

Data Structures and Algorithms for Problem Solving (CS1.304)

Mid-Sem Exam (22 Sep, 2026)

Puzzle 1 (Unstable marriage). Let us recall the stable marriage problem. There are n boys and n girls. Each boy has his preference list over the n girls, and each girl has her preference list over the n boys. An unstable pair is defined as follows:

Definition 1. Let b be a boy and g be a girl. After n marriages have been carried out, the pair (b, g) is said to be unstable if:

- b and g are not married to each other.
- b prefers g more than his current wife.
- g prefers b more than her current husband.

Puzzle 1.1 [8 marks]. In class, we studied that the Gale-Shapley algorithm finds a stable set of marriages in $O(n^2)$ time. In this puzzle, your task is to find an unstable set of marriages in the same (or better) time. The input provided to you is:

- **BOYS**, a 2-dimensional array of size $n \times n$. The boys are named 0 to $n-1$. For every $0 \leq i \leq n-1$, boy i 's most preferred girl is $\text{BOYS}[i][0]$, next comes $\text{BOYS}[i][1]$, and so on, $\text{BOYS}[i][n-1]$ is his least preferred girl.
- **GIRLS**, a 2-dimensional array of size $n \times n$. The girls are named 0 to $n-1$. For every $0 \leq j \leq n-1$, girl j 's most preferred boy is $\text{GIRLS}[j][0]$, next comes $\text{GIRLS}[j][1]$, and so on, $\text{GIRLS}[j][n-1]$ is her least preferred boy.
- **M**, a 1-dimensional array of size n , initialised to $M[i] = i$ for each $0 \leq i \leq n-1$. This **M** will also be your final output.

Can you find an unstable set of n marriages (assume $n \geq 3$) in $O(n^2)$ time? Can you do it in $O(n)$ time? Or even better? Your solution will be graded on the basis of your algorithm, its explanation, and its runtime (the quicker, the better). You can even write plain English sentences in your code/pseudocode, but its description, explanation, and analysis should be clear, coherent, correct, and complete. Store your final answer in the array **M**. That is, if you set $M[i] = j$, then boy i will marry girl j . If you do not edit **M**, then by default, boy 0 will marry girl 0, boy 1 will marry girl 1, and so on, boy $n-1$ will marry girl $n-1$. **Hint:** For a set of n marriages to be unstable, it is enough to find only one unstable pair.

Puzzle 1.2 [2 marks]. The above algorithm was for all $n \geq 3$. Let us now look at $n = 2$ (two boys and two girls). In this puzzle, your task is to devise the preference lists of the four people in such a way that every possible set of marriages amongst them is always stable (that is, an unstable pair never exists, no matter how they get married). Denote your answer in the form of two 2-dimensional arrays **BOYS** and **GIRLS** (as defined earlier), each of size 2×2 . Also explain your answer.

Puzzle 2 (Median heap). In a binary heap (min-heap), we want to maintain the minimum entry in a scenario where entries are being inserted and deleted. As studied in class, the time complexity of basic heap operations in a heap of size n is as follows. (Note that the same time complexities also apply for max-heaps.)

- **FINDMIN** (output the minimum entry present in the heap): $O(1)$.
- **DELETEMIN** (delete the minimum entry from the heap): $O(\log_2 n)$.
- **INSERT(x)** (insert a new and distinct entry x into the heap): $O(\log_2 n)$.

In this puzzle, your task is to maintain the **median** entry in a scenario where entries are being inserted and deleted. The median of a set of n entries is defined as the $\lceil \frac{n}{2} \rceil^{\text{th}}$ smallest entry (assume all entries are distinct). For example, the median of $\{26, 1, 9, 2, 8\}$ is 8, and the median of $\{7, 4, 17, 9, 6, 23\}$ is 7. The time complexities of basic operations should be:

- **FINDMEDIAN** (output the median entry present in the heap): $O(1)$.
- **DELETEMEDIAN** (delete the median entry from the heap): $O(\log_2 n)$.
- **INSERT(x)** (insert a new and distinct entry x into the heap): $O(\log_2 n)$.

Puzzle 2.1 [10 marks]. Design a data structure **median heap** to perform these operations. Just like min-heaps/max-heaps, you can use arrays to implement median heaps. For your algorithms, you can even write plain English sentences in your codes/pseudocodes, but their descriptions, explanations, and analyses should be clear, coherent, correct, and complete.
